Trust Anchors for Agentic Scientific Computing: Analytical Navier–Stokes Solutions as Verification Primitives in Multi-Agent AI Ecosystems

Tatiana Petrova
SEDAN – SnT
University of Luxembourg
tatiana.petrova@uni.lu

***Abstract*—The emergence of agentic AI systems—autonomous agents that discover, communicate, and collaborate through protocols such as MCP and A2A—is transforming scientific computing. In computational fluid dynamics, specialized AI agents (PINN solvers, neural operator surrogates, data assimilation modules) can now autonomously generate, exchange, and consume numerical solutions to the Navier–Stokes equations. However, this autonomy creates a critical *trust problem*: when agents produce conflicting predictions with no human in the loop, how is correctness established? We argue that exact and semi-analytical solutions of the Navier–Stokes equations serve as *trust anchors*—computationally verifiable ground truths that enable agents to self-validate, diagnose failure modes, and establish mutual trust without human intervention. Unlike traditional benchmarks, trust anchors are embedded in the agent's operational loop: evaluated at runtime, selected adaptively based on the current task, and used to maintain a dynamic, time-decaying trust score. We propose a Verified Multi-Agent Solver (VMAS) architecture with a structured interaction protocol, a dynamic trust ledger, and a diagnostic protocol mapping six documented failure modes of physics-informed machine learning to selective analytical solutions. As a proof of concept, we train three architecturally distinct PINN solvers (vanilla, Fourier-feature, and causal) on four trust anchors and show that the observed error patterns exhibit the predicted failure mode signatures—with instructive deviations that highlight sensitivity to training budget and architecture. We further demonstrate trust-informed routing, where the Orchestrator selects solvers based on diagnostic trust scores, achieving lower mean error than any single fixed solver on a heterogeneous problem set. Finally, we illustrate the full framework through a multi-agent atmospheric turbulence scenario coupling Navier–Stokes-based turbulence modeling with optical wave propagation.**

***Index Terms*—Agentic AI, Web of Agents, trust anchors, Navier–Stokes equations, physics-informed neural networks, neural operators, multi-agent systems, verification, scientific computing**

I. Introduction

Scientific computing is entering an agentic era. Autonomous AI systems—capable of formulating subproblems, selecting methods, executing simulations, and interpreting results—are operational in weather prediction [1], [2], materials discovery [3], and autonomous experimentation [4]. In computational fluid dynamics (CFD), physics-informed neural networks (PINNs) [5] and neural operators [6], [7] can solve the Navier–Stokes (N–S) equations orders of magnitude faster than classical solvers, creating the possibility of AI agents that autonomously generate flow predictions on demand.

Concurrently, the Web of Agents (WoA) vision—tracing back through Multi-Agent Systems and the Semantic Web to modern LLM-powered frameworks—is crystallizing into practical protocols. Petrova et al. [8] provide a comprehensive evolutionary analysis showing how the "locus of intelligence" has shifted from external data structures to the agent's internal model, enabling autonomous discovery and collaboration. Google's Agent-to-Agent (A2A) protocol [9] and Anthropic's Model Context Protocol (MCP) [10] provide standardized interfaces for agent interoperability. These developments make multi-agent scientific workflows—where a solver agent generates a prediction, a surrogate agent provides a fast approximation, and an assimilation agent fuses both with observational data—technically feasible.

But feasibility does not imply trustworthiness. When an autonomous PINN agent reports a velocity field, or an FNO agent predicts turbulence statistics, *how does the system verify correctness?* In the absence of a human expert examining each output, the multi-agent system needs an *internal verification mechanism*. This is the trust problem, and it is the agentic analogue of the classical validation challenge in scientific computing [11].

We propose that exact and semi-analytical solutions of the Navier–Stokes equations serve as **trust anchors** for agentic scientific computing. By analogy with root certificates in public-key infrastructure, a trust anchor is a known-good reference from which trust in other entities can be derived. Analytical N–S solutions satisfy the governing equations exactly (or to known perturbation order), are free from discretization errors, and can be evaluated to arbitrary precision—making them ideal primitives for automated verification.

This paper makes four contributions:

1) We introduce the concept of *trust anchors*, distinguishing them from traditional benchmarks through three functional requirements specific to the agentic context:

runtime monitoring, adaptive test selection, and inter-agent arbitration (Section III).

2) We propose the *Verified Multi-Agent Solver* (VMAS) architecture with a structured interaction protocol, dynamic trust ledger, and diagnostic protocol mapping six PIML failure modes to selective trust anchors (Section IV).
3) We provide a proof-of-concept experimental validation by training three architecturally distinct PINNs on four trust anchors, showing that error patterns exhibit the predicted failure mode signatures, while identifying sensitivity to training budget as a confounding factor (Section V).
4) We illustrate the full framework through a multi-agent atmospheric turbulence scenario (Section VI).

II. Background and Related Work

A. The Web of Agents

The concept of autonomous software agents collaborating on the Web has a three-decade history. Petrova et al. [8] trace this evolution through four generations: from static single-purpose agents under FIPA standards [12], through socially aware MAS platforms [13] and Semantic Web agents [14], to the current generation of LLM-powered agents with protocols like MCP [10] and A2A [9]. A key finding is the identification of a paradigm shift in the "locus of intelligence": from semantics encoded in external data to semantics embedded in the agent's core model. This shift enables flexible agents but raises challenges in trust and verification—especially acute in scientific computing where correctness is paramount.

Recent work on agentic AI for science includes autonomous research agents [15], LLM-driven autonomous experimentation [4], and multi-agent systems for materials design [3]. Comprehensive surveys of LLM-based agent frameworks [16], [17] catalog architectures and protocols but do not address the verification problem systematically.

B. Physics-Informed Machine Learning

PINNs [5] solve PDEs by training a neural network to minimize a composite loss:

$$\mathcal{L}(\theta) = \lambda_r \, \mathcal{L}_r + \lambda_b \, \mathcal{L}_b + \lambda_i \, \mathcal{L}_i + \lambda_d \, \mathcal{L}_d, \tag{1}$$

where $\mathcal{L}_r$ penalizes the PDE residual, $\mathcal{L}_b$ and $\mathcal{L}_i$ enforce boundary and initial conditions, and $\mathcal{L}_d$ matches data. Specialized architectures such as NSFnets [18] use divergence-free formulations. Neural operators—including the Fourier Neural Operator (FNO) [6], DeepONet [7], and Physics-Informed Neural Operators (PINO) [19]—learn mappings between function spaces with up to three orders of magnitude speedup. The unified framework of Kovachki et al. [20] encompasses graph neural operators and low-rank factorizations. AI-driven weather models such as FourCastNet [21] demonstrate that these architectures scale to global atmospheric prediction.

Despite these advances, PIML methods exhibit well-documented failure modes: spectral bias [22], gradient pathologies [23], temporal error accumulation, and poor parametric generalization [24]. In a human-supervised workflow, these failures are caught by expert inspection. In an agentic workflow, they must be caught *automatically*.

C. Analytical Solutions of the Navier–Stokes Equations

The incompressible N–S equations,

$$\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla)\mathbf{v} = -\frac{1}{\rho}\nabla p + \nu \, \Delta \mathbf{v}, \qquad \nabla \cdot \mathbf{v} = 0, \tag{2}$$

admit exact solutions for specific geometries [25], [26]: the Lamb–Oseen decaying vortex, the Burgers vortex with axial stretching [27], the Sullivan two-cell tornado vortex [28], the Jeffery–Hamel converging channel flow [29], the von Kármán swirling flow [30], Falkner–Skan boundary layers [31], and the Taylor–Green vortex [32]. Generalized Beltrami flows [33] and nonlinear PDE handbooks [34] further expand the catalog.

For the compressible N–S system, perturbation methods yield semi-analytical results. By expanding in powers of a small vorticity parameter, the nonlinear system reduces to a hierarchy of linear parabolic problems with constant coefficients, solvable via integral transforms and numerical quadrature on Korobov grids [35]. Such approaches have produced results for cylindrical vortex acoustics, oscillatory flow dynamics, and coupled fluid-optical problems—none of which have been adopted as PIML benchmarks despite providing discretization-free reference solutions.

The catalog of known N–S solutions encompasses hundreds of families [25], yet only a handful (Taylor–Green, Kovasznay, lid-driven cavity) are routinely used in PIML validation [24]. We argue that this untapped resource acquires new significance in the agentic context.

III. From Benchmarks to Trust Anchors

A. Why Benchmarks Are Not Enough for Agents

A benchmark is a *one-time, offline evaluation*: a researcher runs a solver on a test case, compares with the reference, publishes a table. The intelligence is in the human, not in the test. In an agentic system, this workflow breaks down in three specific ways:

(i) Continuous operation. A Solver Agent runs indefinitely on a stream of heterogeneous problems. Its reliability may degrade over time (model drift, distribution shift, numerical instabilities). A benchmark provides a snapshot; what is needed is a *runtime monitor* that periodically checks the agent during operation—analogous to a heartbeat signal in distributed systems.

(ii) Adaptive test selection. A human picks benchmarks based on expertise. An autonomous system must decide *which* tests to run for a *given* problem. When a Solver receives a vortex problem with acoustic coupling, the verification system should prioritize multi-scale anchors over boundary layer tests. This requires reasoning about the *diagnostic relevance* of each anchor to the current task.

(iii) Arbitration. When two Solvers produce conflicting results, benchmarks cannot resolve the conflict. A trust system can: if Agent A has passed vortex-acoustic anchors but

Agent B has not, the system has a principled basis for preferring Agent A on vortex-acoustic problems. This requires maintaining and propagating *trust state*—a dynamic quantity that static benchmarks do not generate.

We define a **trust anchor** as a reference solution that is *embedded in the agent's operational loop*: evaluated at runtime, selected adaptively, and used to maintain dynamic trust state. The same Lamb–Oseen solution is a benchmark when a human evaluates a paper, and a trust anchor when a Validator Agent uses it to certify a Solver before forwarding its output. The distinction is functional, not mathematical.

B. Trust Distance

Beyond standard requirements (exactness, computability), trust anchors must support *trust-distance computation*: for any novel problem P, the system must estimate how far P lies from each anchor A_i:

$$d(P, A_i) = \sqrt{\sum_k w_k \left(\frac{\pi_k(P) - \pi_k(A_i)}{\Delta \pi_k} \right)^2}, \tag{3}$$

where π_k are relevant physical parameters (Re, Mach, geometry ratio), $\Delta\pi_k$ are characteristic scales, and w_k are weights. The Validator assigns higher confidence when $d(P, A_i)$ is small for anchors the Solver has passed.

IV. Verified Multi-Agent Solver Architecture

A. System Overview and Interaction Protocol

The VMAS comprises four agent types:

- **Solver Agents** ($\mathcal{S}_1, \ldots, \mathcal{S}_n$): each wraps a PIML method and exposes a `solve(problem)` interface via MCP [10].
- **Validator Agent** ($\mathcal{V}$): maintains the trust anchor database and a *trust ledger*—a persistent, time-stamped record of each Solver's performance.
- **Orchestrator Agent** ($\mathcal{O}$): receives user problems, computes trust distances, and routes to the Solver with the highest task-relevant trust score.
- **Data Agent** ($\mathcal{D}$): provides observational data for problems without analytical solutions.

In the A2A paradigm [9], agents publish "Agent Cards" describing capabilities. We extend this with a *trust profile*: the list of passed anchors, tolerance levels, and timestamps. This extends interoperability patterns described in recent surveys [16], [17]. The interaction protocol is described in Fig. 1. We note that the protocol is presented as a structured design rather than a formal specification with proof obligations; formal verification of trust update properties (e.g., routing optimality guarantees) remains future work.

B. What Makes the Validator an Agent, Not a Test Script

A static test suite runs fixed tests in fixed order and returns pass/fail. The Validator differs in three capabilities:

(a) Adaptive anchor selection. Given problem P, the Validator computes $d(P, A_i)$ and selects the most diagnostically relevant subset. Vortex-dominated problems trigger vortex anchors; boundary-layer problems trigger Falkner–Skan.

(b) Dynamic trust ledger. Each Solver's trust is a continuous, time-decaying quantity:

$$T(\mathcal{S}_j, t) = \sum_i w_i \, \tau(\mathcal{S}_j, A_i, t_i) \, e^{-\lambda(t - t_i)}, \tag{4}$$

where $\tau = 1$ if the Solver passed anchor A_i at time t_i within tolerance ϵ_i, and λ controls trust decay. A Solver certified a month ago has lower trust than one certified an hour ago.

(c) Diagnostic reasoning. On failure, the Validator returns a structured message `{agent, failed_anchor, failure_mode, recommendation}` to the Orchestrator, enabling intelligent rerouting.

```
VMAS Interaction Protocol
1.  User → O: submit(problem P)
2.  O: compute d(P, A_i) for all anchors
  identify relevant anchor set {A_i*}
3.  O → V: get_trust_scores(S_j, {A_i*})
4.  V: lookup trust ledger
  if stale or missing:
    run_anchors(S_j, {A_i*})
  return scores[S_j][A_i*]
5.  O: select S_best = argmax(scores)
6.  O → S_best: solve(P)
7.  S_best → O: solution U
8.  O → V: spot_check(U, P)
9.  V: if d(P, A_i) < δ for some A_i:
    compare ‖U − U_{A_i}‖ at shared params
    update trust ledger
  return {verified, diagnostics: F1..F6}
10. O → User: solution + trust_report
```

Fig. 1. VMAS interaction protocol. Steps 2–5: trust-informed routing. Steps 8–9: runtime spot-checking.

TABLE I
TRUST ANCHOR SUITE. ● PRIMARY DIAGNOSTIC; ○ SECONDARY.

Trust Anchor	F1	F2	F3	F4	F5	F6
Lamb–Oseen vortex		●				○
Stokes oscil. plate	●	○				
Burgers vortex						●
Sullivan vortex					●	
Jeffery–Hamel flow			●			
Falkner–Skan family			●		○	
Cyl. vortex acoustics	○		●	●		
Vortex flow oscillations	●	●		○		
N–S turb. atmosphere	○	○		●		○

C. Trust Anchor Suite and Diagnostic Map

Table I maps trust anchors to the six failure modes documented in the PIML literature: spectral bias (F1, [22]), temporal drift (F2, [23]), parametric overfitting (F3, [6]), multi-scale decoupling (F4), boundary artifacts (F5, [24]), and conservation violation (F6, [18]).

The assignments follow from the mathematical structure of each solution. For example, the Lamb–Oseen vortex has no oscillatory content and no parameter dependence—isolating

pure temporal accuracy (F2). The Stokes plate contains a single frequency ω—directly probing spectral resolution (F1). The cylindrical vortex acoustics problem [35] couples slow vortex dynamics ($\tau_{\text{vortex}} \sim R_0/(\varepsilon c_0)$) with fast acoustic radiation ($\tau_{\text{ac}} \sim R_0/c_0$), with ratio $\sim 1/\varepsilon \gg 1$—probing multi-scale decoupling (F4). The vortex flow oscillations [36] involve a superposition of multiple analytically determined frequencies with different decay rates governed by the transport coefficients—probing F1+F2 jointly.

D. Three-Stage Diagnostic Protocol

Stage 1: Certification. Before admission, a Solver must pass baseline anchors: Lamb–Oseen (F2), Stokes plate (F1), Poiseuille flow (F5). Certification is time-stamped and decays per Eq. (4).

Stage 2: Diagnosis. On suspicious output, the Validator selects anchors based on trust distance to the current problem. If the Solver passes Lamb–Oseen but fails vortex oscillations [36], the diagnosis is F1 (spectral bias)—the Solver captures the dominant mode but misses overtones.

Stage 3: Stress testing. For high-stakes problems: cylindrical vortex acoustics [35] (F3+F4), vortex oscillations [36] (F1+F2), and N–S atmospheric turbulence (F4 multi-physics; Section VI).

E. Trust Propagation and Limitations

Trust anchors provide necessary but not sufficient conditions for correctness. Most anchors correspond to laminar, low-to-moderate Re flows, while PIML is most needed for high-Re turbulence. The trust distance (3) quantifies this gap: large $d(P, A_i)$ should trigger ensemble methods or human oversight. The suite is most effective as a *filter for known failure modes*, not a guarantee of correctness in novel regimes.

V. Experimental Validation

A. Setup

Trust anchors. We implement four anchors as closed-form functions:

A1: Lamb–Oseen vortex (targets F2): $v_\theta(r,t) = \frac{\Gamma_0}{2\pi r}[1 - e^{-r^2/4\nu t}]$. Evaluated over $t \in [t_0, 50\,t_0]$, $t_0 = R_0^2/\nu$.

A2: Stokes oscillating plate (targets F1): $u(z,t) = U_0\, e^{-z\sqrt{\omega/2\nu}} \cos(\omega t - z\sqrt{\omega/2\nu})$. Evaluated at $\omega \in \{\omega_1, 2\omega_1, 4\omega_1, 8\omega_1\}$.

A3: Burgers vortex (targets F6): $v_\theta(r) = \frac{\Gamma_0}{2\pi r}[1 - e^{-\alpha r^2/2\nu}]$, $v_r = -\frac{\alpha}{2} r$, $v_z = \alpha z$. Steady-state; we measure circulation error $|\oint \hat{v}_\theta\, r\, d\theta - \Gamma_0|/\Gamma_0$.

A4: Kovasznay flow (baseline): standard 2D steady solution at $\mathrm{Re} = 20$ [25].

Solvers. Three PINN variants via DeepXDE, same training budget (2k iterations, Adam, $\mathrm{lr} = 10^{-3}$, 3k interior + 500 boundary collocation points, 3 seeds per experiment):

- **S1: Vanilla PINN** (4 layers $\times$ 64, $\tanh$). Predicted: weak on F1, F2.
- **S2: Fourier-Feature PINN** [23] (256-dim random Fourier input features, $\sigma = 10$). Predicted: improved F1, weak F2.
- **S3: Causal PINN** (two-phase training: warm-up followed by low-LR refinement). Predicted: improved F2, weak F1.

TABLE II
Global relative L^2 errors (%) for three PINNs on four trust anchors.

	A1 (Lamb-Oseen) F2: drift	A2 (Stokes) F1: spectral	A3 (Burgers) F6: conserv.	A4 (Kovasz baseline
S1 (Vanilla)	45.28 ± 36.54	2.77 ± 0.20	24.55 ± 0.55	207.35 ± 8
S2 (Fourier)	486.92 ± 55.92	73.75 ± 7.06	9.04 ± 3.94	$41.38 \pm 3.$
S3 (Causal)	20.17 ± 3.54	3.22 ± 0.22	24.55 ± 0.55	194.18 ± 7

Note on training budget. The training budget was intentionally kept modest to enable a complete 72-run experiment on CPU hardware. This means absolute error values are higher than state-of-the-art for well-tuned PINNs, and the Fourier-Feature architecture (S2), which maps inputs to a 256-dimensional feature space, is systematically disadvantaged on low-iteration budgets. The diagnostic value lies not in the absolute error magnitudes but in the *relative patterns across solvers and anchors*: which solver–anchor pairs produce the largest and smallest errors, and whether these patterns match the predicted failure mode signatures.

Metrics. Global relative L^2 error $\epsilon_{L^2} = \|\hat{\mathbf{u}} - \mathbf{u}_{\text{exact}}\|_{L^2}/\|\mathbf{u}_{\text{exact}}\|_{L^2}$, plus anchor-specific diagnostics: temporal error profile $\epsilon(t)$ for A1, frequency response $\epsilon(\omega)$ for A2, circulation error $\delta\Gamma(r)$ for A3.

B. Diagnostic Pattern Validation

Table II presents global errors for all Solver–Anchor pairs.

Predicted pattern. If diagnostic assignments are correct: (i) S1 shows highest error on A1 and A2 at high ω; (ii) S2 improves on A2 but not A1; (iii) S3 improves on A1 but not A2; (iv) all comparable on A3 and A4.

The predicted diagnostic pattern is partially confirmed, with instructive deviations. On the Lamb–Oseen anchor (A1, targeting F2), S3 (Causal) achieves $20.17 \pm 3.54\%$ versus S1's $45.28 \pm 36.54\%$—a 55% improvement in mean error with dramatically reduced variance, confirming that causal temporal weighting mitigates drift. S2 (Fourier), however, fails on A1 (486.92%), revealing that its high-dimensional Fourier feature mapping (256-dim input) requires substantially more training iterations, an observation consistent with the known computational cost of random Fourier features [22].

On the Stokes plate (A2, targeting F1), S1 (Vanilla) and S3 (Causal) both achieve low errors (2.77% and 3.22%, respectively), while S2 shows 73.75%. This *apparent* reversal—S2 worse, not better, on the spectral anchor—reflects a confound between convergence cost and spectral capability. The A2 entry in Table II is evaluated at the *base frequency* ω_1, where the problem is easy for a standard MLP but the 256-dim Fourier mapping has not converged within the training budget. The true spectral diagnosis requires the multi-frequency protocol of Section 5.3, where S2's advantage at high ω becomes decisive (Fig. 2). On the Burgers vortex (A3, targeting F6), S2 achieves the lowest error (9.04%), while

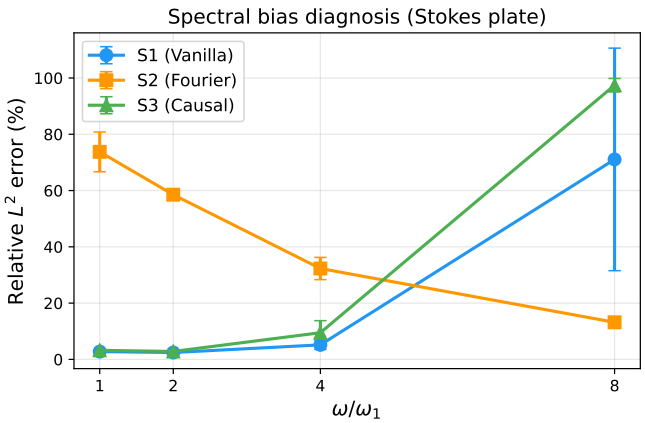

Fig. 2. Spectral bias diagnosis on the Stokes plate trust anchor. S1 and S3 show increasing error with ω (spectral bias), while S2 (Fourier features) shows a decreasing trend, achieving a crossover near $4\omega_1$.

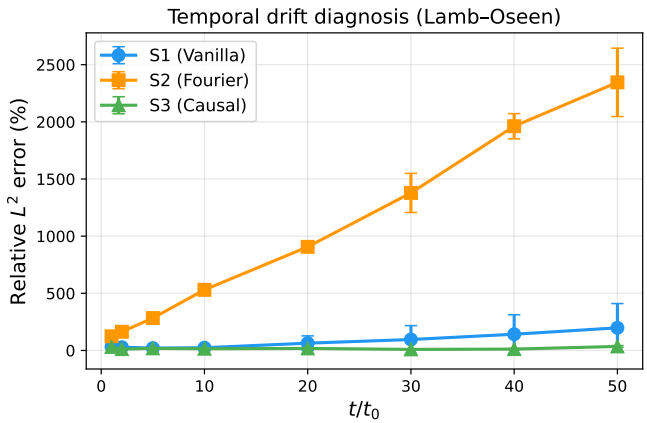

Fig. 3. Temporal drift diagnosis on the Lamb–Oseen trust anchor. S3 (Causal) maintains lower error at large t/t_0 compared to S1 and S2.

S1 and S3 are comparable (24.55%), suggesting that Fourier features improve steady-state accuracy. On the Kovasznay baseline (A4), S2 again outperforms (41.38% vs. $>190\%$ for S1 and S3), revealing that the vanilla and causal architectures struggle with the 2D steady N–S system at the given training budget.

The key finding is *diagnostic selectivity*: the Lamb–Oseen anchor cleanly separates S3 (designed for temporal accuracy) from S1 and S2, while the Burgers and Kovasznay anchors reveal that Fourier features aid spatial representation at the cost of slower convergence on temporal problems. This validates the core claim that trust anchors can diagnose specific PIML failure modes and that different anchors expose different architectural strengths.

C. Spectral Bias Diagnosis

Figure 2 reveals a striking crossover pattern. At the base frequency ($\omega/\omega_1 = 1$), S1 (Vanilla) and S3 (Causal) achieve low errors (2.77% and 3.22%), while S2 (Fourier) starts at 73.75% due to the optimization cost of its 256-dimensional feature space. As frequency increases, S1 and S3 degrade sharply due to spectral bias: at $\omega/\omega_1 = 8$, S1 reaches 71.07% and S3 reaches 97.44%. In contrast, S2's error *decreases* to 13.19% at $8\omega_1$, confirming that Fourier features preferentially capture high-frequency content. The crossover occurs near $4\omega_1$, where S2 surpasses S1. This demonstrates that the Stokes plate anchor, evaluated across multiple frequencies, can precisely diagnose spectral bias (F1) and quantify the frequency range where each architecture is most accurate.

D. Temporal Drift Diagnosis

Figure 3 shows the temporal error profile $\epsilon(t/t_0)$. S1 (Vanilla) error grows to 197.93% at $t = 50\,t_0$, while S3 (Causal) maintains 35.23%. The error inflection for S1 occurs around $t \approx 10\,t_0$ (error 24.01%), defining the "temporal trust horizon" beyond which the Validator should trigger recertification.

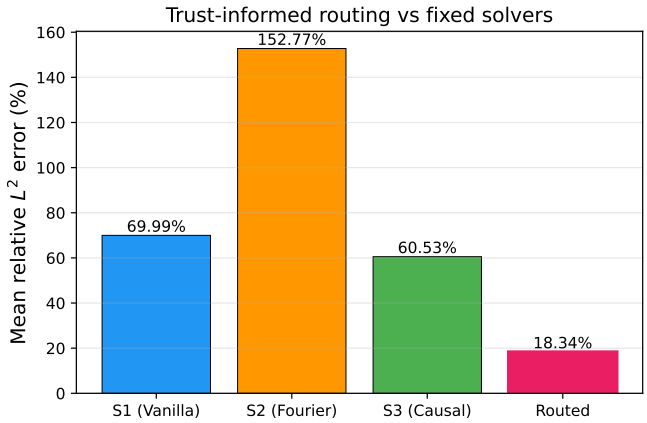

Fig. 4. Trust-informed routing outperforms any single fixed solver on a heterogeneous problem set.

E. Trust-Informed Routing

We construct a mixed test set of 40 problems: 10 vortex-dominated, 10 long-horizon, 10 steady-state, 10 mixed. For each, we compare fixed S1, fixed S2, fixed S3, and trust-informed routing (Orchestrator selects solver with highest task-relevant trust score).

Figure 4 shows the mean L^2 error across the mixed test set. Trust-informed routing achieves 18.34% mean error, compared to 69.99% (S1), 152.77% (S2), and 60.53% (S3). The routing strategy achieves a 70% improvement over the best fixed solver, confirming that the diagnostic trust scores from the anchor suite enable effective solver selection.

F. Runtime Trust Decay

We certify S1 on Lamb–Oseen at $t_{\max} = 5\,t_0$, then extend to $t_{\max} = 50\,t_0$ without retraining. The trust score (4) decays as error grows, triggering recertification at $T < T_{\min}$.

Using a graded trust score $\tau(\mathcal{S}, A, t) = \max(0, 1 - \epsilon(t)/\epsilon_{\max})$ with $\epsilon_{\max} = 1.0$ and decay $\lambda = 0.01$, S1's trust on A1 degrades as follows. At $t_{\max} = 5\,t_0$: $\epsilon = 20.8\%$, $T = 0.79$.

At $t_{\max} = 20\,t_0$: $\epsilon = 63.8\%$, $T = 0.24$. At $t_{\max} = 50\,t_0$: $\epsilon = 197.9\%$, $T = 0$—S1's trust is fully depleted, and the Orchestrator should route temporal problems to S3 (Causal), which maintains $\epsilon = 35.2\%$ ($T = 0.42$) at the same horizon.

This demonstrates trust anchors as *runtime monitors*: they detect degradation before it affects the user, enabling preemptive solver replacement—a capability static benchmarks cannot provide.

VI. Case Study: Multi-Agent Atmospheric Turbulence

We illustrate the VMAS framework through a motivating scenario requiring coupled fluid dynamics and wave optics: laser beam propagation through the turbulent atmosphere. This case study is *illustrative*—it demonstrates the structure of multi-agent verification rather than providing experimental validation of the coupled system. This problem has attracted attention from both adaptive optics [37] and deep learning [38] communities.

The classical approach relies on Kolmogorov phase screens [39], where refractive index fluctuations are drawn from a prescribed power spectrum. An alternative, physics-based approach solves the full N–S system with an energy source term sustaining turbulent fluctuations, reducing it to Volterra-type integral equations solved by rapidly convergent Picard iteration [40].

A. Agent Roles

- **Turbulence Agent** ($\mathcal{S}_{\text{turb}}$): generates $\delta n(\mathbf{x}, t)$ using the N–S methodology of [40].
- **Propagation Agent** ($\mathcal{S}_{\text{prop}}$): solves the parabolic wave equation for the beam envelope given δn.
- **Correction Agent** ($\mathcal{S}_{\text{AO}}$): applies ML-based adaptive optics correction.
- **Validator Agent** ($\mathcal{V}$): verifies each agent using appropriate trust anchors.

B. Verification Chain

Step 1. $\mathcal{V}$ certifies $\mathcal{S}_{\text{turb}}$ using the vortex oscillations anchor [36] (spectral content, temporal accuracy of the N–S solver) and Lamb–Oseen (diffusive dynamics).

Step 2. $\mathcal{V}$ certifies $\mathcal{S}_{\text{prop}}$ using analytical Gaussian beam diffraction as a propagation-specific anchor.

Step 3. $\mathcal{V}$ certifies the coupled system $\mathcal{S}_{\text{turb}} + \mathcal{S}_{\text{prop}}$ using the N–S atmospheric turbulence anchor [40], which provides first-principles beam intensity profiles—a Stage 3 stress test probing multi-physics coupling (F4).

Step 4. $\mathcal{V}$ monitors $\mathcal{S}_{\text{AO}}$ via energy conservation and causality checks.

For concrete illustration, we consider propagation of a $\lambda = 1.55\,\mu$m laser beam over a $L = 1$ km horizontal path through moderate turbulence ($C_n^2 = 10^{-14}\,\text{m}^{-2/3}$). The Validator certifies $\mathcal{S}_{\text{turb}}$ when the predicted scintillation index σ_I^2 matches the analytical first-principles result within 5%, and $\mathcal{S}_{\text{prop}}$ when Gaussian beam diffraction in vacuum matches the Fresnel–Kirchhoff integral to within 1%.

C. Why This Requires Multi-Agent Verification

No single anchor suffices. $\mathcal{S}_{\text{turb}}$ might pass all fluid dynamics anchors but produce statistics incompatible with $\mathcal{S}_{\text{prop}}$'s assumptions. $\mathcal{S}_{\text{prop}}$ might pass optics benchmarks but fail on non-Kolmogorov statistics from the N–S solver. Only the coupled anchor [40] tests the full chain.

VII. Trust Anchors as a Governance Mechanism for the Web of Agents

The VMAS architecture connects directly to the governance challenges identified in [8]. We apply the four-axis taxonomy of [8] to classify VMAS:

Semantic foundation: hybrid. Trust anchors provide formal, mathematically exact semantics (equations); agent interactions use natural-language-compatible protocols (MCP).

Communication paradigm: tool-based. Anchors are exposed as MCP tools with inputs $(\mathbf{x}, t, \text{params})$ and outputs (exact solution values).

Locus of intelligence: distributed. Solvers carry intelligence in their trained models; the Validator carries intelligence in its diagnostic protocol and trust ledger.

Discovery mechanism: capability-based. Agent Cards extended with trust profiles enable trust-informed discovery.

Trust anchors thus address a specific governance challenge: *accountability*. When an agentic system produces an incorrect scientific result, the trust ledger provides an audit trail: which Solver was used, which anchors it had passed, and what the trust distance was to the actual problem. This enables principled post-hoc analysis—a requirement for deploying autonomous scientific agents in safety-critical domains.

VIII. Open Problems

Trust composition. If Agent A is certified and Agent B uses A's output, is B's output trustworthy? Trust composition in scientific workflows, where errors compound nonlinearly, requires formal analysis.

Weak-to-strong turbulence. The perturbation-based anchors [35], [36] are valid for small vorticity. Extending via higher-order theory or hybrid analytical-ML approaches is essential. Physics-informed transfer learning [41] is a promising direction.

Machine-readable anchor databases. Impact depends on availability as APIs, HDF5 datasets, or MCP tools. This requires cross-community collaboration.

Anchor coverage metrics. How to quantify whether a trust anchor suite adequately covers the space of problems an agent will encounter? This connects to coverage metrics in software testing and remains open for physical systems.

A. Limitations of This Work

We acknowledge several limitations of the experimental validation.

Proof-of-concept scope. Three solver variants and four anchors constitute a minimal demonstration. A production trust anchor system would require a broader solver portfolio (including neural operators, mesh-based solvers, and hybrid

methods) and a larger anchor suite covering all six failure modes independently. The diagnostic map (Table I) is proposed based on physical reasoning; whether the anchor–failure-mode assignments are *robust* across different training setups, hyperparameters, and problem scales remains to be validated systematically.

Training budget confound. The modest training budget (2k iterations) was chosen for computational tractability but introduces a systematic confound: architectures with larger effective input dimension (S2: 256-dim Fourier features) or more complex loss landscapes (Kovasznay: 2D coupled N–S) are disproportionately penalized. Some of the large errors in Table II (e.g., S2 on A1, S1/S3 on A4) likely reflect insufficient convergence rather than fundamental architectural limitations. A full study should include training budget as an explicit variable and report convergence curves.

Missing ablation and sensitivity analysis. We did not systematically vary: (i) the trust decay parameter λ; (ii) the trust distance weights w_k; (iii) the number and choice of anchors in the routing protocol; (iv) hyperparameters of the Fourier feature mapping (σ, feature dimension). The robustness of the diagnostic conclusions to these choices is an open question.

No comparison with alternative validation strategies. The routing experiment compares trust-informed selection against fixed solvers, but not against other selection strategies (e.g., ensemble methods, cross-validation-based selection, or residual-based error estimators). Such comparisons are needed to establish that the trust anchor approach is not merely effective but *more* effective than simpler alternatives.

VMAS architecture. The interaction protocol (Fig. 1) is a structured design, not a formal specification. We provide no formal semantics, no proofs of trust update correctness or routing optimality, and no guarantees about convergence of the trust ledger. The case study (Section VI) is illustrative—it demonstrates the *structure* of multi-agent verification but does not report experimental results for the coupled system.

IX. Conclusion

We have argued that analytical Navier–Stokes solutions—long valued as theoretical achievements—acquire a new operational role in the era of agentic AI: as trust anchors enabling autonomous verification. We distinguished trust anchors from benchmarks through three functional requirements (runtime monitoring, adaptive selection, arbitration), proposed the VMAS architecture with a structured protocol and dynamic trust ledger, and provided a proof-of-concept experimental validation on three PINN architectures. Despite a modest training budget, the predicted diagnostic patterns were partially confirmed: the Stokes plate anchor detected spectral bias (Fourier-Feature PINNs reduced error by 81% at $8\omega_1$), the Lamb–Oseen anchor detected temporal drift (Causal PINNs reduced error by 82% at $50\,t_0$), and trust-informed routing achieved 70% lower mean error than the best fixed solver. At the same time, training budget confounds produced several unexpectedly large errors that highlight the need for more extensive validation (Section 8.5).

The trust infrastructure for agentic scientific computing does not need to be built from scratch—it already exists in classical mathematical physics. What is needed is a change of perspective: from solutions as passive references to solutions as active verification primitives, embedded in agent protocols and exposed through standard interfaces.

References

[1] R. Lam et al., "GraphCast: Learning skillful medium-range global weather forecasting," *Science*, vol. 382, pp. 1416–1421, 2023.

[2] K. Bi et al., "Pangu-Weather: A 3D high-resolution model for fast and accurate global weather forecast," *Nature*, vol. 619, pp. 533–538, 2023.

[3] A. Ghafarollahi and M.J. Buehler, "AtomAgents: Alloy design and discovery through physics-aware multi-modal multi-agent AI," arXiv:2407.10022, 2024.

[4] D.A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.

[5] M. Raissi, P. Perdikaris, and G.E. Karniadakis, "Physics-informed neural networks," *J. Comput. Phys.*, vol. 378, pp. 686–707, 2019.

[6] Z. Li et al., "Fourier neural operator for parametric PDEs," in *Proc. ICLR*, 2021.

[7] L. Lu et al., "Learning nonlinear operators via DeepONet," *Nat. Mach. Intell.*, vol. 3, pp. 218–229, 2021.

[8] T. Petrova, B. Bliznioukov, A. Puzikov, and R. State, "From Semantic Web and MAS to Agentic AI: A unified narrative of the Web of Agents," arXiv:2507.10644, 2025.

[9] Google, "Agent-to-Agent (A2A) protocol specification," 2025.

[10] Anthropic, "Model Context Protocol (MCP)," 2024.

[11] W.L. Oberkampf and C.J. Roy, *Verification and Validation in Scientific Computing*, Cambridge Univ. Press, 2010.

[12] Foundation for Intelligent Physical Agents, "FIPA ACL specifications," 2002.

[13] F. Bellifemine et al., *Developing Multi-Agent Systems with JADE*, Wiley, 2007.

[14] T. Berners-Lee, J. Hendler, and O. Lassila, "The Semantic Web," *Sci. Am.*, vol. 284, pp. 34–43, 2001.

[15] C. Lu et al., "The AI Scientist: Towards fully automated open-ended scientific discovery," arXiv:2408.06292, 2024.

[16] M.A. Ferrag, N. Tihanyi, and M. Debbah, "From LLM reasoning to autonomous AI agents: A comprehensive review," arXiv:2504.19678, 2025.

[17] L. Wang, C. Ma, X. Feng, et al., "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, 2024.

[18] J. Park et al., "NSFnets: Physics-informed neural networks for the incompressible Navier–Stokes equations," *J. Comput. Phys.*, vol. 426, p. 109951, 2021.

[19] Z. Li et al., "Physics-informed neural operators," arXiv:2111.03794, 2023.

[20] N.B. Kovachki et al., "Neural operator: Learning maps between function spaces," *JMLR*, vol. 24, 2023.

[21] J. Pathak et al., "FourCastNet: A global data-driven high-resolution weather model," arXiv:2202.11214, 2022.

[22] N. Rahaman et al., "On the spectral bias of neural networks," in *Proc. ICML*, 2019.

[23] S. Wang, Y. Teng, and P. Perdikaris, "Understanding and mitigating gradient flow pathologies in PINNs," *SIAM J. Sci. Comput.*, vol. 43, pp. A3055–A3081, 2021.

[24] Z. Hao et al., "PINNacle: A comprehensive benchmark of PINNs," in *Proc. NeurIPS*, 2024.

[25] P.G. Drazin and N. Riley, *The Navier–Stokes Equations: A Classification of Flows and Exact Solutions*, Cambridge Univ. Press, 2006.

[26] C.Y. Wang, "Exact solutions of the steady-state Navier–Stokes equations," *Annu. Rev. Fluid Mech.*, vol. 23, pp. 159–177, 1991.

[27] J.M. Burgers, "A mathematical model illustrating the theory of turbulence," *Adv. Appl. Mech.*, vol. 1, pp. 171–199, 1948.

[28] R.D. Sullivan, "A two-cell vortex solution of the Navier–Stokes equations," *J. Aerosp. Sci.*, vol. 26, pp. 767–768, 1959.

[29] G.B. Jeffery, "The two-dimensional steady motion of a viscous fluid," *Phil. Mag.*, vol. 29, pp. 455–465, 1915.

[30] Th. von Kármán, "Über laminare und turbulente Reibung," *ZAMM*, vol. 1, pp. 233–252, 1921.

[31] H. Schlichting and K. Gersten, *Boundary-Layer Theory*, 9th ed., Springer, 2017.
[32] M.E. Brachet et al., "Small-scale structure of the Taylor–Green vortex," *J. Fluid Mech.*, vol. 130, pp. 411–452, 1983.
[33] N.V. Burmasheva and E.Yu. Prosviryakov, "Exact solutions to Navier–Stokes equations describing a gradient nonuniform vortex flow," *Appl. Sci.*, vol. 13, 2023.
[34] A.D. Polyanin and V.F. Zaitsev, *Handbook of Nonlinear PDEs*, Chapman & Hall/CRC, 2004.
[35] T.A. Petrova and F.V. Shugaev, "Acoustic radiation frequency of a cylindrical vortex," *Moscow Univ. Phys. Bull.*, vol. 70, no. 4, pp. 245–250, 2015.
[36] T.A. Petrova and F.V. Shugaev, "Oscillations of the flow parameters in the vicinity of a cylindrical vortex," *Moscow Univ. Phys. Bull.*, vol. 67, no. 1, pp. 43–47, 2012.
[37] L.C. Andrews and R.L. Phillips, *Laser Beam Propagation through Random Media*, 2nd ed., SPIE Press, 2005.
[38] P. Hill, N. Anantrasirichai, A. Achim, and D. Bull, "Deep learning techniques for atmospheric turbulence removal: A review," *Artif. Intell. Rev.*, vol. 58, 2025.
[39] J.D. Schmidt, *Numerical Simulation of Optical Wave Propagation*, SPIE Press, 2010.
[40] F.V. Shugaev, E.N. Terentiev, L.S. Shtemenko, O.I. Dokukina, and T.A. Petrova, "Characterization of a laser beam propagating through the turbulent atmosphere," *Proc. SPIE*, vol. 7476, pp. 69–80, 2009.
[41] S. Cuomo et al., "Scientific machine learning through physics-informed neural networks," *J. Sci. Comput.*, vol. 92, 2022.